

DAA LAB ASSIGNMENT-5

1. QUICK SORT:

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void swap(int *a, int *b) {
```

```
    int t = *a;
```

```
    *a = *b;
```

```
    *b = t;
```

```
}
```

```
int partition(int a[], int low, int high) {
```

```
    int pivot = a[high];
```

```
    int i = low - 1;
```

```
    for (int j = low; j < high; j++) {
```

```
        if (a[j] <= pivot) {
```

```
            i++;
```

```
        swap(&a[i], &a[j]);  
    }  
}
```

```
    swap(&a[i + 1], &a[high]);  
    return i + 1;  
}
```

```
void quickSort(int a[], int low, int high) {  
    if (low < high) {  
        int pi = partition(a, low, high);  
        quickSort(a, low, pi - 1);  
        quickSort(a, pi + 1, high);  
    }  
}
```

```
int main() {  
    int n;  
    scanf("%d", &n);  
  
    int *a = (int *)malloc(n * sizeof(int));
```

```
if (!a) {  
    printf("Memory allocation failed\n");  
    return 1;  
}
```

```
for (int i = 0; i < n; i++)  
    scanf("%d", &a[i]);
```

```
quickSort(a, 0, n - 1);
```

```
for (int i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
free(a);  
  
return 0;  
}
```

OUTPUT:

```
(doniraj@doni)-[~]  
$ nano quick.c  
  
(doniraj@doni)-[~]  
$ gcc quick.c -o quick  
  
(doniraj@doni)-[~]  
$ ./quick  
Enter number of elements: 5  
Enter elements:  
3  
11  
5  
2  
9  
Sorted array:  
2 3 5 9 11
```

2.MERGE SORT

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void merge(int a[], int left, int mid, int right) {
```

```
    int i, j, k;
```

```
    int n1 = mid - left + 1;
```

```
int n2 = right - mid;
```

```
int L[n1], R[n2];
```

```
for (i = 0; i < n1; i++)
```

```
    L[i] = a[left + i];
```

```
for (j = 0; j < n2; j++)
```

```
    R[j] = a[mid + 1 + j];
```

```
i = 0; j = 0; k = left;
```

```
while (i < n1 && j < n2) {
```

```
    if (L[i] <= R[j])
```

```
        a[k++] = L[i++];
```

```
    else
```

```
        a[k++] = R[j++];
```

```
}
```

```
while (i < n1)
```

```

        a[k++] = L[i++];
    while (j < n2)
        a[k++] = R[j++];
}

void mergeSort(int a[], int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(a, left, mid);
        mergeSort(a, mid + 1, right);

        merge(a, left, mid, right);
    }
}

```

```

int main() {
    int n;

    printf("Enter number of elements: ");

    scanf("%d", &n);
}

```

```
int a[n];  
  
printf("Enter elements:\n");  
  
for (int i = 0; i < n; i++)  
    scanf("%d", &a[i]);  
  
mergeSort(a, 0, n - 1);  
  
printf("Sorted array:\n");  
  
for (int i = 0; i < n; i++)  
    printf("%d ", a[i]);  
  
return 0;  
  
}
```

OUTPUT:

```
(doniraj@doni)-[~]  
$ nano merge.c  
  
(doniraj@doni)-[~]  
$ gcc merge.c -o merge  
  
(doniraj@doni)-[~]  
$ ./merge  
Enter number of elements: 6  
Enter elements:  
12  
1  
5  
7  
3  
9  
Sorted array:  
1 3 5 7 9 12
```

3)BST

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
struct Node {
```

```
    int data;
```

```
    struct Node *left, *right;
```

```
};
```

```
struct Node* createNode(int value) {
```

```

    struct Node* node = (struct Node*)malloc(sizeof(struct Node));

    node->data = value;

    node->left = node->right = NULL;

    return node;
}

struct Node* insert(struct Node* root, int value) {

    if (root == NULL)

        return createNode(value);

    if (value < root->data)

        root->left = insert(root->left, value);

    else if (value > root->data)

        root->right = insert(root->right, value);

    return root;
}

void inorder(struct Node* root) {

    if (root != NULL) {

        inorder(root->left);

        printf("%d ", root->data);

        inorder(root->right);
    }
}

```

```

    }
}

int main() {

    struct Node* root = NULL;

    int n, value;

    printf("Enter number of nodes: ");

    scanf("%d", &n);

    printf("Enter values:\n");

    for (int i = 0; i < n; i++) {

        scanf("%d", &value);

        root = insert(root, value);

    }

    printf("Inorder traversal: ");

    inorder(root);

    return 0;

}

```

OUTPUT:

```
(doniraj@doni)-[~]  
$ nano bst.c  
  
(doniraj@doni)-[~]  
$ gcc bst.c -o bst  
  
(doniraj@doni)-[~]  
$ ./bst  
Enter number of nodes: 5  
Enter values:  
3  
11  
8  
5  
9  
Inorder traversal: 3 5 8 9 11  
  
(doniraj@doni)-[~]  
$
```

DONIRAJ.SALINDRA

CH.SC.U4CSE24245